SharingApp - Code Smell Review

Part 1 | Two distinct smells | Proposed improvements only

1. Duplicated Code

In AddItemActivity.saveItem() (AddItemActivity.java:54) and EditItemActivity.saveItem() (EditItemActivity.java:127), the same six form values are read and checked for empty strings, using almost identical branches and error messages. AddContactActivity.saveContact() and EditContactActivity.saveContact() also repeat email-format and username-availability checks. These are repeated implementations of the same validation policies, so this is Duplicated Code, independently of how long the methods are. A rule or error-message change must be applied in several places; overlooking one copy can make equivalent forms behave inconsistently and increases maintenance and regression-testing work. A proposed fix is to extract shared item/contact validation components that return named field errors, while each Activity remains responsible for displaying errors on its widgets. Creation versus editing can be explicit validation options so that an unchanged username remains valid during editing. The common validation component should preserve the intended check order and messages. This is a proposal only: Part 2 introduces separate validateInput() methods and does not implement this shared-validator redesign.

2. Data Clumps

The length_str, width_str and height_str values travel together through AddItemActivity.saveItem() and EditItemActivity.saveItem(), then through ItemController.setDimensions(String, String, String) (ItemController.java:48) and Item.setDimensions(String, String, String) (Item.java:82). The latter immediately packages the three values into a Dimensions object. The recurring tuple represents one cohesive physical concept, but the Activity/controller boundary repeatedly exposes it as three independent strings; this is Data Clumps. Swapping width and height still produces valid Java code, repeated positional argument lists obscure intent, and changes to dimension representation or units must be coordinated across these boundaries. A proposed fix is to use the existing Dimensions value object throughout the controller/model API: construct one validated Dimensions instance at the form boundary and pass that single object to the controller. Dimension-specific invariants and formatting can then live with that value object instead of being distributed among string arguments. Merely renaming the parameters would not remove the clump. This API redesign is proposed only and is not applied in Part 2.